JWT and SPRING SECURITY

Our User class must implement UserDetails ,UserDeails is a contract or a standard blue print defined by Spring Security

think of it as a standard ID card format

we make our User class implement the UserDetails , we promise to present our user's information in this standard format that Spring Security can understand

Our User class might have all fields like email,pwd and first Name

,Spring Security has no idea what those are. But when we implement those methods from UserDetails

When Spring Security calls getUsername(), then we tell is , "Take value from my Email field "

When it calls getPassword() , we say take it from my pwd field

when it calls getAuthorities() we say Here are the user roles like USER or ADMIN

UserDetails acts as an adapter or bridge . It connects our custom User object , with its unique field names ,to the standard system that spring security uses for authentication and authorization . It translates data into language Spring Security speaks

```
@Override
public Collection<? extends GrantedAuthority> getAuthorities()
{
    return List.of(new SimpleGrantedAuthority(role.name()));
}
```

This method must return a collection of like list of permissions

GrantedAuthority : Official Spring boot name for single permission . Think of this as authority the user has been granted .Most often used to represent a role ROLE_USER or ROLE_ADMIN

role.name() : refers to the field inside the User class , the enum that can be USER or ADMIN

new SimpleGrantedAuthority("ADMIN") : The role is wrapped in Simple Granted Auth object.

The jjwt library

all of the heavylifting is done by the io.jsonwebtoken library

think of it as a highly specialized secure factory that knows how to build and inspect high tech security badges like jwts . JwtService class is the manager . We don't build the badges ourselves we give the JwtService Manager class the information like the users name and an expiration date and tell it to build one for it or it to inspect one

JwtSigningKey is used ,it is like a invisible tamper proof ink used to print watermark on badge .

1 Generating new token

```
private String generateToken(Map<String,Object>
extraClaims,UserDetails userDetails)
{
    return Jwts.builder()
            .setClaims(extraClaims)
            .setSubject(userDetails.getUsername())
            .setIssueAt(new Date(System.currentTimeMillis()))
            .setExpiration(new
Date(System.currentTimeMillies()+1000*60*24))

.signWith(getSigningKey(),SignatureAlgorithm.HS256)
            .compact();
}
```

Jwts.builder() : get ready to build a badge

.setClaims(extraClaims) : Here is some extra data to put on the badge

in intial call this is empty

.setSubject(userDetails.getUsername()) : we say “ Set the subject of this badge to misbah@abc.com . The subject is the user to whom the token belongs

.setIsueedAt(..) : Stamp the badge with the current time and the date it was created

.setExpiration(…) : Stamp the badge with an expiration date of 24 hours from now , tokens don’t live forever

.signWith(getSigningKey(), SignatureAlgorithm.HS256):

calls our private helper method getSigningKey() to get secret key .

Helper method :

private Key getSigningKey()

byte[]keyBytes=Decoders.BASE64.decode(jwtSigningKey)

Our key in application.properties is a string. Cryptography works in bytes so this line decoded our string key into a list of bytes

return Keys.hmacShaKeyFor(keyBytes) : they key that we recenty turned into bytes and stord into keyBytes variable

we pass these bytes to the jjwt factory and say create an official cryptographically secure key object

HMAC-SHA256 : it is a message authentication code that uses the SHA-256 Hash function to provide both data integrity and authentication . It combines a secret Key with the message data before hashing it with SHA – 256

A method to check both

1 Integrity -> Has the message been changed

2 Authenticity -> Does it come from someone who knows the secret key?

SHA-256 secure Hash Algorithm with a 256 bit output

A Hashing function that turns data into fixed length fingerprint

What is Hashing

in hash function we put any size data a word a book a file , and it gives back a fixed size fingerprint (SHA 256 ,its always 64 hex characters long

even If we change one letter the fingerprint changes significantly

input : cat output : SHA-256 (d077f244def8a….

input : cats output 29drt06mw3…

Hashing alone isn't enough : Hashing can tell if something changed but cannot prove who created it

We send a file to a friend , We send its SHA-256 hash separately

But hacker can change the file recalculate the hash , replace both file and hash

Introducing Secret Key

It is like a password that only you and your friend know

we use it so that even if someone sees the msg they cannot create the fingerprint without key

what HMAC does:

1 Takes our secret key

2 takes the msg

3 mix them in a way

4 hash the result using SHA-256

hash again with padding tricks

and we get a special fingerprint

JWT

Jason Web Token : It I s a string like user id roles expiration time plus signature to ensure it is not tampered with

A JWT has three parts each BASE64 encoded

header.payload.signature

header: how the token is made

payload: user info ,like your id and expiring time

signature : like a wax to detect changes

NEED OF BASE64

we write in JSON

{"userId": 123, "role": "admin"}

it has spaces brackets quotes etc , computers can read it but when sending in URLS or HTTPS headers , its better to convert it into a safe text format

That’s where Base 64 encoding comes in

Base64 turns any data into lettrs , numbers no spaces or wird symbols and is safe for sending over in URLS or HTTPS

hello - > BASE64->SGVsbG8

{"userId":123} → Base64 → eyJ1c2VySWQiOjEyM30=

Its NOT encryption – anyone can decode BASE64

its just packaging data neatly

BUILDING A JWT

Lets make a token for

{"userId": 123, "role": "admin"}

STEP 1 create header

Header says what algorithm is used to sign the token

{“alg”: “HS256”, “typ” : “JWT”}

BASE64 : eyJhbGciOiAiSFMyNTYiLCAidHlwIjogIkpXVCJ9

STEP 2 create PAYLOAD

Payload is your actual data :

{"userId": 123, "role": "admin"}

its BASE64 :

eyJ1c2VySWQiOiAxMjMsICJyb2xlIjogImFkbWluIn0=

STEP 3 create SIGNATURE

signature =HMAC(“mysecret” , <header>.<payload>”)

then base64 encodes the signature as well

WE get to our final JWT

<header_base64>.<payload_base64>.<signature_base64>

eyJhbGciOiAiSFMyNTYiLCAidHlwIjogIkpXVCJ9.eyJ1c2VySWQiOiAxMjMsICJyb2xlIjogImFkbWluIn0=.abc123xyz

header :

{

“alg”:HS256”,

“typ”:”JWT

}

HS256 : HMAC with SHA-256 is used to sign the token

How signing works :

Create header JSON and encode with Base64

header_b64=base64(header_json)

payload_b64=base64(payload_json)

header_b64+ “.”+base64

Sign with HMAC SHA 256 using servers secret key

signature : HMAC_SHA256(secret_key.header_b64+

Attach signature :

So JWT takes header that contains info about what algorithm is going to be used for hashing and the type (JWT)

then payload which is the actual content the data that we are using

and then signature that has the secret key which will be used for signing the header and payload

secret key :

it is not stored in JWT

its not sent to the client

its used only by the server to generate and verify the signature

secret_key= “my super secret 123”

A random string of letters or numbers , or even binary data but it must be kept safe

How HMAC SHA 456 use it for signing

Data to sign

When a JWT is created the header and payload are Base64 encoded and joined

“<header_base64>.<payload_base64>”

Example :

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjEyMywicm9sZSI6ImFkbWluIn0

Step B – Sign with HMAC-SHA256

signature = HMAC_SHA256(secret_key, "<header>.<payload>")

what HMAC does :

Takes secret key like “my secret key”

takes the message that is “<header>.<payload>”

Mixes them using HMAC formula

Key is padded & XORed with constants

Message is hashed with SHA-256 along with key

This process ensures only someone with the exact key can produce the same output

Final Token

Signature is then Base64 encoded and appended to the header and payload

<header_base64>.<payload_base64>.<signature_base64>

So JWT takes three parts the header which describes the type of authentication i.e jwt to show that it is a jwt token and the algorithm hmac sha 256 that is to be used

message contains the actual data content that we are going to send and signature that uses secret key and using that secret key it takes header and payload and does signing on it using hmac formula is pads the key with xor sha 256 is used for hashing the msg and key and after the signature part is hashed it is base64 encoded too

key is padded to a fixed size

XORred with 2 constants

Hashed with SHA256

Hashed again with SHA256

JwtService class is the part of app that

creates the JWTs with a header payload and signature

it also reads and validates JWTs make sure they are real and not expired

JWT uses HS256

1 Signing Key

```
@Value("${token.signing.key}")
private String jwtSigningKey;
```

this gets the secret key from application.properties ,private pwd we use in H,AC SHA256 signing

Key is never sent to client only the server has it

2 Extract Username from token

```
public String extractUserName(String token) {
    return extractClaim(token, Claims::getSubject);
}
```

Subject in JWT means the main user identifier usually the username or user id

this method opens the token ,reads the payload (middle of JWT)

returns whatever is stores as sub in the payload

3 Generating a token

```
public String extractUserName(String token) {
    return extractClaim(token, Claims::getSubject);
}
```

creates a token for a logged in user

uses a helper method that allows extra claims extra data if u want to add more than just the username

4 Validating a token

```
public boolean isTokenValid(String token, UserDetails userDetails) {
    final String userName = extractUserName(token);
    return (userName.equals(userDetails.getUsername())) &&
!isTokenExpired(token);
}
```

it extract username from token

Compare with username from from your database (userdetails)

check if the token is expired

this works because signature proves it hasn’t been expired

5 Extracting a specific claim

private <T> T extractClaim(String token, Function<Claims, T> claimsResolvers) {

final Claims claims = extractAllClaims(token);

return claimsResolvers.apply(claims);

}

Claim = data unside the payload part of JWT

6 Generate Token

```
private String generateToken(Map<String,Object> extraClaims,UserDetails
userDetails)
{
    return Jwts.builder()
            .setClaims(extraClaims)
            .setSubject(userDetails.getUsername())
            .setIssueAt(new Date(System.currentTimeMillis()))
            .setExpiration(new
Date(System.currentTimeMillies()+1000*60*24))
            .signWith(getSigningKey(),SignatureAlgorithm.HS256)
            .compact();
}
```

how this ties for JWT theory

setClaims extracts the payload data

setSubject extracts the sub:username

setIssuedAt extracts the issuing time “iat”:now

setExpiration sets the expiration time

signWith : specifiy the key to sign with we get it from our method

compact convert to string “<header>.<payload>.<signature>”

Header = set automatically

Payload = extraClaims + username + issueAt + expiration

Signature = created by HMAC-SHA256

When the server US gets a JWT from the client

it is split into header ,payload ,signature

recalculate the signature using our own the servers signature key

compare it with given signature

if they match then -> then toke is valid

if they don't -> then reject data may be tampered

All heavy lifting is done with the help of jwt

it builds the header

we just say

.signWith(key, SignatureAlgorithm.HS256)

JJWT automatically makes

{

"alg" : "HS256",

"typ" : "JWT"

}

when we call .sognWith(getSigningKey(),,SignatureAlgorithm.HS256

the jjwt library sees and understands that we want to use HS256 algorithm , so it adds {"alg: "HS256"}

Since we are using Jwts.builder() , the library knows we are building standard JWT so it adds the second part {"typ":"JWT"}

payload json will look something like

{"sub": "janedoeexample", "iat":1723402205, "exp" :1723488605}

The signature :

using secret key header and payload is combined

in the code it is handled with .signWith() and .compact()

Events

header and payload json ins created and encoded with base64UrlEncode(header)

AuthenticationService

This class handles user sign up (create + save user , give JWT) and sign in (check creds give JWT)

UserRepository talks to the the DB to save/find users

PasswordEncoder – hashes the users password before saving

JwtService jwtService – creates JWTs

AuthenticationManager authenticationManager :

@Service makes this a spring service bean @RequiredArgsConstructor makes a constructor for all final fields so Spring can inject them

```
public JwtAuthenticationResponse signup(SignUpRequest request)
{
    var user = User.builder()
            .firstName(request.getFirstName())
            .lastName(request.getLastName())
            .email(request.getEmail())
            .password(passwordEncoder.encode(request.getPassword()))
            .role(Role.USER).build();
    userRepository.save(user);
    var jwt = jwtService.generateToken(user);
    return JwtAuthenticationResponse.builder().token(jwt).build();

}
```

We get a SignUpRequest from client containing firstName , lastName , email and pwd . Build a user entity from those values

password encoder uses bcrypt hashing to encode that pwd , we never save plain text pwd

userRepo.save(user) stores the user in the db

jwtService.generateToken(user) creates JWT for that user (JWT will contain subject / claims and be signed using our servers secret key)

This method will return JwtAuthenticationResponse with token usually a JSON like {“token” : “eyjWiqhrns/….”}

LOGIN : User providesusername and pwd (plain text)

THIS IS WHERE AuthenticationManager steps in

when we hit /login

we create Authentication object like UsernamePasswordAuthenticationToken with username and plain pwd from the request

this is passed to the AuthenticationManager

AuthenticationManager is an interface , its default implementation is the ProviderManager . It doesn’t check pwds itself

Instead it delegates task to one or more AuthenticationProviders

AuthenticationProvider

Authentication authenticate(Authentication authentication) throws AuthenticationException;

In a username pwd setup main provider is the DaoAuthenticationProvider :

It knows how to call userDetailsService to fetch the user from DB by username

and

Takes the plain text password hashes it with bcrypt encoder and compares it with the one in db

UserDetailsService loads the UserDetails object (username , hashed pwd,roles)

it does not compare pwds just loads the data

the DaoAuthenticationProvider gets this UserDetails aand then calls PasswordEncoder.matches(plain_pwd,hashed_pwd)

if matches -> authentication success return a fully populated Authentication object

it not throw BadCredentialsException

My understanding

out /login is hit with creds

SPRING SECURITY WRAPS them in an Authentication object called UsernamePasswordAuthenticationToken

AuthenticationManager gets this token

authenticationManager an interface whose implementation is the ProviderManager handles further , it doesn't actually check instead it delegates the task to its worker DaoAuthenticationProvider , it uses UserDetailsService to retrieve user data like uname ,pwd and role

it calls PasswordEncoder as PasswordEncoder.matches(plain_pwd,hashed_pwd) and if success sends a populated Authentication object

FLOW IN DETAIL

Spring Security login filter (Username PasswordAuthenticationFilter intercepts it and it takes the username and password and wraps them into and Authentication object but with no authorities yet

Authentication authRequest = new UsernamePasswordAuthenticationToken(username,password);

at this stage

principal = username

creds=plain pwd

authorities=null

authenticated=false

AuthenticationManager an interface whose default implementation is ProviderManager doesn't check pwds itself .instead it loops through a list of AuthenticationProovider and asks if anyone amongs its workers know how to handle this type of token

DaoAuthenticationProvider takes over ,it is the built in provider for uname/pwd authentication

UserDetails userDetails=userDetailsService.loadUserByUsername(username);

It must return a UserDetails object that includes username,hashed password and authorities (roles/permission)

DaoAuthenticationProvider calls

passwordEncoder.matches(plain_pwd,hashed_pwd)

plain password is hashed using that exact same salt and cost factor

compares the newly computed hash to stored hash

if they are identical pwd is correct

a fully populated Authnetication object is returned

the provider returns a new Authentication object UsernamePasswordAuthenticationToken

principal = UserDetails(full object)

creds= null

authorities =[ROLE_USER,ROLE_ADMIN,..]

authenticated=true

it says user is verified there who they are an dhwat they can do

Spring security Context stores this , It puts this Authentication object into SecurityContextHolder

and so for the rest of the request (and future requests) we get the logged in user

SecurityConfiguration in config folder

This is a special java class that will configure and override Spring Securitys default behaviour . This is where we will tie together all the components that we have built UserRepository PasswordEncoder and JwtService and tell Spring Security exactly how to use them

This file will be the brain of our system and without having it Spring Security will configure just about anything and everything with a default login page and sessions , but this is not what we want for our JWT based API , we need to explicitly tell it

which endpoints and public and which ones are private , which ones to protect

to be stateless and to not use sessions

how to find users

how to check for passwords

how to check for our custom JwtAuthenticationFilter before its own filters

AuthenticationManager decides whether a given authentication request (username /pwd ,token ,etc) is valid – it either returns a fully authenticated Authentication Object or throws an AuthenticationException

AuthenticationManager is just an interface it has one main method authenticate(Authentication authentication)

AuthenticationProvider is the component that actually checks credentials . It has two responsible

support(Class <?> authentication)- say what token types it can process (username/pwd,api tokens,SAML,etc)

authenticate(Authentication authentication) – perform the real check : verify password , validate token signature

Dao AuthenticationProvider is the provider Spring uses for the typical username/pwd flows – It calls UserDetailsService to load user data and uses a PasswordEncoder to compare pwd hashes

Spring default implementation is the ProviderManager it doesn't authenticate directly it just keeps a list of AuthenticationProvider instances and delegates work to them

WHAT happens when a login request arrives

1. A filter UsernamePasswordAuthenticationFilter extracts username/password from the request and builds an unauthenticated UsernamePasswordAuthenticationToken
2. The filter call authentication.authenticate(token)
3. ProviderManager receives the token and eiterates its AuthentciationProviders to find one that supports that token
4. The chosen provider DaoAuthenticationProvider loads the user via UserDetailsService checks the password via PasswordEncoder it either returns a fully authenticated Authentication object with authorities success or it returns BadCredentialsException
5. The filter receives the result and on success sets the security context so the request is not authenticated

JwtAuthenticationFilter

Our SecurityConfiguration expects a JwtAuthenticationFilter , but we haven't created it yet.

We are creating a filter class that will run for every single request coming into our application , this filter will be our active gatekeeper

This is the component that actually checks for and validates the JWT on incoming requests .Its job is to

intercept a request

Check if JWT is in the Authorization header

if it does then validate the token using JwtService

If token in valid , update spring security's context to tell it that the user in authenticated for this request

Filter is the internal actual guard who stands at the entrance and physically scans

This filter runs once per HTTP request , extracts a JWT token from the Authorization : Bearer<token> header (if present) , validates it ,loads the user , builds a Spring Authentication object , puts SecurityContext (so request is treated as authenticated whether true or false but yes it is authenticated)

then continues filter chain

extends OncePerRequestFilter

it ensures the filter executes once pre request (not multiple times on forwards)

Injected Dependencies

JwtService jwtService is our helper that

reads the token

extracts the claim (like subject /username)

validates the signature and expiry (isTokenValid)

builds Key from Base64 secret , etc

UserDetailsService userDetailsService your DB backed loader that returns UserDetails (uname,hashed pwd and authorities that user has)

```
final String authHeader=request.getHeader("Authorization");
```

Reads the standard header where clients send tokens :

Authorisation: Bearer eyJhbGciOiJI….

```
if(authHeader==null ||  !authHeader.startsWith("Bearer"))
{
filterChain.dofilter(…);
return;
}
```

If header is missing or doesn't start with Bearer skip JWT logic and let request continue , it keeps public endpoints working and avoids errors when client don't send a token

```
jwt=authHeader.substring(7);
```

Removes Bearer prefix and now jwt is compact JWT string

header.payload.signature

```
userEmail=jwtService.extractUserName(jwt);
```

calls JwtService which typically base64 decoded payload

reads the sub subject claim or specific username claim

returns an email

This is fast check , it doesn’t yet assert that token ins valid (signature expiry) it just reads the username claim

if(userEmail !=null && SecurityContextHolder.getContext()

if userEmail !=null means we found a subject in that token and SecurityContextHolder.getContext().getAuthenticated()==null

prevents re authenticating if some earlier filter already put an Authentication there ,(it was already authenticated)

this avoids unnecessary work and prevents overwriting a valid authentication

UserDetails userDetails = this.userDetailsService.loadByUsername(userEmail);

it loads user form DB , the returned UserDetails typically contains

username /email

hashedpwd

GrantedAuthority list(roles)

Because we usually want to the authorities roles from the DB and also to verify the token belongs to an actual (or implement account disabled checks etc)

if(jwtService.isTokenValid(jwt, userDetails)) {…}

This is the real validation

isTokenValid should verify the token signature using your signing key HMAC SHA256

confirm token expiry is in the future

check that subject equals userDetails.getUsername() or check other claims like token ID

if any check fails do not authenticate the request

Build UsernamePasswordAuthenticationToken

JwtAuthenticationFilter authToken=new JwtAuthenticationFilter(userDetails,null ,userDetails.getAuthorities());

This is Spring Authentication implementation used to represent authenticated principal

principal userDetails object (so controllers /security expressions can access the username and or roles)

creds=null because we don't need the the pwd during the token authentication

authorities : userDetails.getAuthorities() -> roles /permissions used by @PreAuthorize @Secured etc

authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

Adds a web related metadata (IP address ,session id) into the authToken .Useful for logging or security checks

SecurityContextHolder.getContext().setAuthenticated(authToken)

revisiting the method prototype again

@Override

protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull HttpServletResponse response,

@NonNull FilterChain filterChain)

throws ServletException, IOException {

@Override : here we are letting it be known that we are overriding a method from the parent class OncePerRequestFilter in this case

OncePerRequestFilter defines a method called doFilterInternal() which once called per HTTP request (as opposed to standard servlet filters that can run multiple times per request if the request is forwarded

Method Parameters

HttpServletRequest request

represents the incoming HTTP request

lets you read request headers ,query params , body , method (GET/POST) path ,etc

request.getHeader("Authorization") could give you the JWT token from the request

HttpServletResponse response

Represents the HTTP response being sent back to client

Lets us set header,status cods and response body

response.setStatus(HttpServletResponse.SC_UNAUTHORIZED) if auth fails

FilterChain filterChain

represents the chain of filters that req will go through

Spring Security sets up filter chain where each filter can :

perform some logic auth ,logging ,CORS handling etc

Call filterChain.doFilter(request,response) to pass the request to the next filter

or stop the chain send a 401 response immediately if auth fails

How this fits with other classes

SecurityConfiguration adds this filter before UsernamePasswordAuthenticationFilter : so token auth happens early and bypasses form login

JwtService:

extractUserName(jwt) -> used here to get userEmail

isTokenValid(jwt,userDetails) -> verifies signature and expiry (uses are loaded (DB via userRepository.findByEmail(…))

AuthenticationController

creating actual API endpoints

we will create the classes that define api endpoints like /signup and /signin

These controllers listen for incoming HTTP requests and delegate the actual work to our AuthenticationService

@RestController

lets spring know that this class is a controller that handles HTTP Requests

Every method returns data not views

the @responseBody behaviour is implicit -Spring converts return objects like JwtAuthenticationResponse

@RequestMapping("/api/v1/auth")

it sets the base URL path for all methods

/signup becomes /api/v1/auth/signup

It is a common practice in REST APIS to version routes v1,v2 so future updates don't break existing clients

@RequiredArgsConstructor

From Lombok automatically generates a constructor that accepts all final fields

since AuthenticationService is maked final , Spring will auto inject it into the controller without having to write @Autowired

DEPENDENCY INJECTION

AuthenticationService is where the real business logic occurs , creating new users validating credentials generating JWT tokens

Signup endpoint POST

w are creating a new resource (a user account)

signup involves sending sensitive data

@RequestBody SignUpRequest request

Spring takes JSON body of the request and map it to a SignUpRequest object

{

"email": test@example.com,

"password": "myp",

"firstname" : "john",

"lastname" : "doe"

}

becomes :

SignUpRequest req = new SignUpRequest (“test@example.com”, “myp” , “john” , “doe”);

authService.signup(req)

The service likely :

1. validates that the email is not already taken
2. 2 encrypts pw using bcrypt
3. saves user in the db
4. generates a jwt token for immediate login after signup

returns a JwtAuthenticationResponse containing the token

ResponseEntity.ok(…)

Wraps the service response into HTTP 200 OK

Signin endpoint

the key difference is that this one

authenticates an existing user instead of creating one

request body contains only username and password

it loads user by email

verifies the password that was stored in hash

generates jwt token if creds are valid

returns token in JwtAuthenticationResponse

How this fits with Jwt flow

/signup

user sends detail -> gets jwt in response

client stores jwt

signin -> user sends creds gets jwt if correct

accessing protected endpoints

accessing protected endpoints

client sends jwt in authorization : bearer <token > header

JwtAuthenticationFilter intercepts request validates token sets securityContextHolder for that thread

JwtAuthenticationFilter

after client recievs token from signin /signup , it must send it with each protected request

Authorization : Bearer <JWT>

Extract token from header ,

jwtService.extractUsername(token) -> username/email

loads user details ,

jwtService .isTokenValid (token ,userDetails) ->verifies signature + expiry

builds usernamePasswordAuthenticationToken : puts it in SecurityContextHolder

END TO END FLOW

Client -> POST /api/v1/auth/signup (JSON)

Controller .signup()

->AuthenticationService.signup()

->PasswordEncoder.encode()

->UserRepository.save()

->JwtService.generateToken()

<-JwtAuthenticationResponse {token}

<- 200 OK + token

Sign in

Client ->POST /api/v1/auth/signin (JSON)

Controller.signin()

->AuthenticationService.signin()

->AuthenticationManager.authenticate()

->DaoAuthenticationProvider

->UserDetailsService.loadUserByUsername()

->PasswordEncoder.matches()

->JwtService.generateToken()

->JwtAuthenticationResponse {token }

<- 200 OK {token}

Accessing a private endpoint later

Client -> GET /api/v1/users/me

Header : Authorization : Bearer <Token>

JwtAuthenticationFilter

->extract token

->extract userna,e

-> load UserDetails

->validate token

->set Authentication with Security Context

->set Authentication with authentication principal

<- 200 OK + data

Spring via Jackson deserializes JSON into the DTO

Jackson is the library that handles serialization and deserialization

ser = java object -> JSON

deserlzn=JSON-> Java object

Spring Boot automatically includes Jackson and wires it up so that

incoming request JSON is deserialzed into Java Object

Controller returns objects DTO/Entities are serialized into JSON for the response